

CS321 Programming Language

Task Activity 2

Name: Mark Renier B. Fostanes	Section: BSCS 3A2
-------------------------------	-------------------

Part I – Function Execution Trace

1) Function callback analysis

```
def compute(a, b, func):  
    return func(a, b)  
  
def operation(x, y):  
    return x * y + x  
  
result = compute(3, 4, operation)  
print(result)
```

1. The function compute receives two values and another function. It does not do the operation by itself. It sends a and b to the function stored in func.
2. The callback function is operation because it is passed into compute and called inside it.
3. The output is 15 because $\text{operation}(3, 4) = 3 * 4 + 3 = 12 + 3 = 15$.

Simple rewrite that accepts many operations:

```
def compute(a, b, operations):  
    results = []  
    for op in operations:  
        results.append(op(a, b))  
    return results  
  
def add(x, y):  
    return x + y  
  
def multiply(x, y):  
    return x * y  
  
def custom(x, y):  
    return x * y + x  
  
print(compute(3, 4, [add, multiply, custom]))
```

2) Tuple mutability analysis

```
record = ("Alice", "Bob", 2025)  
record[0].append("Charlie")  
print(record)
```

4. The output is `(["Alice", "Bob", "Charlie"], 2025)`.
5. The tuple allowed the change because the tuple itself was not replaced. The program only changed the list stored inside the tuple.

6. The mutable element is the list ["Alice", "Bob"]. Lists can be changed.
7. This is called shallow immutability. The tuple cannot change its main elements, but if one element is a mutable object, that inner object can still be modified.

Part II – Callback Pipeline Design

8. A simple pipeline solution is shown below. Each callback processes the data one after another.

```
def pipeline(data, callbacks):
    current = data
    print("Start:", current)
    for func in callbacks:
        current = func(current)
        print(func.__name__ + ":", current)
    return current

def square_numbers(data):
    return [x * x for x in data]

def filter_odd(data):
    return [x for x in data if x % 2 != 0]

def subtract_two(data):
    return [x - 2 for x in data]

numbers = [1, 2, 3, 4, 5]
final_result = pipeline(numbers, [square_numbers, filter_odd, subtract_two])
print("Final:", final_result)
```

- Step-by-step result for the example:

```
Start: [1, 2, 3, 4, 5]
square_numbers: [1, 4, 9, 16, 25]
filter_odd: [1, 9, 25]
subtract_two: [-1, 7, 23]
Final: [-1, 7, 23]
```

3) Recursive state analysis

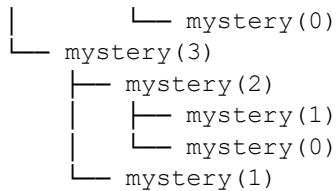
```
def mystery(n):
    if n <= 1:
        return 1
    return mystery(n-1) + mystery(n-2)

print(mystery(5))
```

9. The exact output is 8.

10. Recursive call tree:

```
mystery(5)
├─ mystery(4)
│   ├─ mystery(3)
│   │   ├─ mystery(2)
│   │   │   ├─ mystery(1)
│   │   │   └─ mystery(0)
│   │   └─ mystery(1)
│   └─ mystery(2)
│       └─ mystery(1)
└─
```



11. The total number of recursive calls is 15.
12. This algorithm is inefficient because it repeats the same work many times. For example, `mystery(3)` and `mystery(2)` are called again and again.

4) Decorator analysis

```
def logger(func):
    def wrapper(x, y):
        print("Executing:", func.__name__)
        result = func(x, y)
        print("Result:", result)
        return result
    return wrapper
```

```
@logger
def add(a, b):
    return a + b
```

```
print(add(5, 3))
```

13. The decorator adds extra behavior before and after the original function. Here it prints the function name and its result.
14. Full output:
Executing: add
Result: 8
8
15. The function that replaces `add` during execution is `wrapper`.
16. Modified decorator with execution time:

```
import time

def logger(func):
    def wrapper(x, y):
        start = time.time()
        print("Executing:", func.__name__)
        result = func(x, y)
        end = time.time()
        print("Result:", result)
        print("Execution time:", end - start)
        return result
    return wrapper
```

5) Generator analysis

```
def sequence(n):
    for i in range(n):
        if i % 2 == 0:
            yield i
        else:
```

```
yield i * 2
```

```
g = sequence(5)
for value in g:
    print(value)
```

17. The output sequence is 0, 2, 2, 6, 4.
18. The generator keeps its state by pausing at each yield. When the loop continues, it resumes from the exact place where it stopped.
19. yield gives one value at a time and pauses the function. return ends the function completely and sends back one final value.

<https://www.youtube.com/watch?v=cPKPDpYqlwE>